

UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Electrical Engineering

OPEN ENDED LAB REPORT

Lab: C++ Programming | EE170L

Title:

Student Academic Transcript System Using Structures in C++

Student Name:	Shah khalid
Registration No:	BF25NWELE0672
Section:	B
Submitted To:	Mr. Rizwan
Lab:	EE170L – C++ Programming Lab
Date:	16 May 2026

1. Objective

The objective of this lab is to design and implement a C++ program that manages and displays academic transcripts for multiple students of the B.Sc Electrical Engineering program. The program makes use of structures (struct) to represent both student information and individual course details, allowing for organized storage and retrieval of academic data.

Upon entering a valid registration number, the program searches through the student records and displays a formatted transcript showing the student's name, registration number, enrolled program, course codes, course titles, credit hours, grades, SGPA, and CGPA.

2. Theory / Background

2.1 Structures in C++

A structure (struct) in C++ is a user-defined data type that groups together variables of different data types under a single name. Structures are fundamental to organizing real-world data, such as student records, where each student has attributes of different types (integer, string, float).

Syntax example:

```
struct Course {  
    string code;  
    string title;  
    float CH;  
    string grade;  
};
```

2.2 Nested Structures

A structure can be nested inside another structure. In this program, the Course struct is nested within the Student struct, meaning each student holds an array of Course objects. This is a practical way to represent a real-world relationship where one student is enrolled in multiple courses.

2.3 Arrays of Structures

An array of structures allows the program to store data for multiple students simultaneously. Declaring Student s[10] creates ten Student objects, each containing all the fields defined in the struct, including the nested Course array. Iteration over this array with a for loop enables efficient searching and display of any particular student's record.

2.4 String and iomanip Library

The <string> library enables the use of the C++ string class for storing textual data such as names, course codes, and grades. The <iomanip> library provides stream manipulators such as setw() (set width) and left, which are used to align the transcript output in neat columns — a critical aspect of producing readable formatted output.

2.5 Linear Search

The program implements a simple linear search algorithm. It iterates through all student records, comparing each student's regNo with the user-entered registration number. Once a match is found, the boolean flag 'found' is set to true and the transcript is displayed. If no match is found after the full iteration, a 'Student not found' message is printed.

3. Tools & Environment

Tool / Software	Details
Programming Language	C++ (ISO C++11 or later)
Compiler	GCC / G++ (MinGW on Windows)
IDE	Code::Blocks / Dev-C++ / VS Code
Libraries Used	<iostream>, <iomanip>, <string>
Operating System	Windows 10 / 11

4. Program Source Code

The complete source code of the program is presented below with inline explanation of key sections:

4.1 Header Files and Global Constants

```
#include<iostream>
#include<iomanip>
#include<string>
using namespace std;

const int STUDENTS = 10;
const int SUBJECTS = 8;
```

The header files provide input/output, formatting, and string functionality. Constants STUDENTS and SUBJECTS define the fixed array sizes for the program.

4.2 Structure Definitions

```
struct Course {
    string code;    // Course code e.g. EE170
    string title;   // Course title e.g. C++ Programming
    float CH;       // Credit hours
    string grade;   // Grade obtained e.g. A, B+
};

struct Student {
    int regNo;      // Registration number
    string name;     // Student full name
    string program;  // Degree program
    Course course[SUBJECTS]; // Array of 8 courses (nested struct)
    float SGPA;      // Semester GPA
    float CGPA;      // Cumulative GPA
};
```

4.3 Array Initialization (Sample Records)

```
Student s[STUDENTS] = {
    { 3001, "Muhammad Ali", "B.Sc Electrical Engineering",
      {
        {"BSI111", "Linear Algebra", 3, "A" },
        {"BSI142", "English", 3, "B+"},
        {"BSI110", "Pak Study", 2, "A" },
        {"EE162", "Mechanics", 3, "A" },
        {"EE157L", "Workshop", 1, "A" },
        {"EE170", "C++ Programming", 3, "A" },
      }
    }
};
```

```

        {"EE170L", "Programming Lab",    1, "A+"},
        {"BSI101", "Fahmi Quran",       1, "A" }
    },
    3.88, 3.88 },
    // ... (9 more students) ...
};

```

4.4 Main Function – Search and Display

```

int main() {
    int regNo;
    bool found = false;

    cout << "Enter Registration Number: ";
    cin  >> regNo;

    for (int i = 0; i < STUDENTS; i++) {
        if (regNo == s[i].regNo) {
            found = true;
            cout << "\n===== UET TRANSCRIPT =====\n";
            cout << "Name          : " << s[i].name    << endl;
            cout << "Registration : " << s[i].regNo    << endl;
            cout << "Program      : " << s[i].program << endl;
            cout << left
                 << setw(10) << "Code"
                 << setw(30) << "Subject"
                 << setw(8)  << "CH"
                 << setw(8)  << "Grade" << endl;
            for (int j = 0; j < SUBJECTS; j++) {
                cout << left
                     << setw(10) << s[i].course[j].code
                     << setw(30) << s[i].course[j].title
                     << setw(8)  << s[i].course[j].CH
                     << setw(8)  << s[i].course[j].grade << endl;
            }
            cout << "\nSGPA : " << s[i].SGPA << endl;
            cout << "CGPA : " << s[i].CGPA << endl;
        }
    }
    if (!found) cout << "Student not found";
    return 0;
}

```

5. Program Output

When the program is compiled and executed, the user is prompted to enter a registration number. The following shows the expected output for Registration Number 3004 (Hamza Bilal):

```

Enter Registration Number: 3004

===== UET TRANSCRIPT =====
Name          : Hamza Bilal
Registration   : 3004
Program       : B.Sc Electrical Engineering

Code          Subject                                CH          Grade

```

```

BSI111    Linear Algebra          3      A+
BSI142    English                 3      A
BSI110    Pak Study              2      A
EE162     Mechanics              3      A+
EE157L    Workshop               1      A
EE170     C++ Programming        3      A+
EE170L    Programming Lab        1      A+
BSI101    Fahmi Quran            1      A

```

SGPA : 4.00

CGPA : 4.00

6. Algorithm / Flowchart

The following steps describe the algorithm implemented in the program:

1. START
2. Declare and initialize an array of 10 Student structures, each containing 8 Course structures.
3. Prompt the user to enter a registration number.
4. Iterate through the student array using a for loop (i = 0 to 9).
5. Compare each student's regNo with the entered value.
6. If match found: set found = true, display transcript (name, reg, program, courses, SGPA, CGPA).
7. After loop: if found == false, print 'Student not found'.
8. END

7. Student Data Summary

The following table summarizes the 10 students stored in the program along with their CGPA:

Reg No	Name	Program	SGPA	CGPA
3001	Muhammad Ali	B.Sc EE	3.88	3.88
3002	Ahmed Khan	B.Sc EE	3.25	3.25
3003	Asad Ullah	B.Sc EE	2.80	2.80
3004	Hamza Bilal	B.Sc EE	4.00	4.00
3005	Bilal Saeed	B.Sc EE	2.20	2.20
3006	Usman Yousaf	B.Sc EE	3.50	3.50
3007	Yasir Shah	B.Sc EE	3.90	3.90
3008	Faisal Rehman	B.Sc EE	3.30	3.30
3009	Zeeshan Ali	B.Sc EE	3.10	3.10
3010	Junaid Ahmed	B.Sc EE	3.95	3.95

8. Discussion

This program effectively demonstrates the practical application of C++ structures in modeling real-world academic data. The use of nested structures (Course inside Student) reflects a clean and logical data model that mirrors how university data is actually organized.

The `setw()` and left manipulators from `<iomanip>` are critical for producing aligned, readable output — a skill directly applicable to console-based data reporting systems. The linear search algorithm, while simple, is appropriate for a small fixed dataset. For larger datasets, a more efficient search (binary search or hash map) would be preferred.

A key limitation of the current design is the use of compile-time initialized static data. In a real system, student records would be read dynamically from a file or database. Future improvements could include:

- File I/O to read and write student records from external files.
- Dynamic memory allocation using pointers or vectors.
- Automatic GPA computation from grade inputs instead of hardcoded values.
- A menu-driven interface to add, delete, and update student records.

9. Conclusion

This open-ended lab successfully demonstrates the use of C++ structures to model and manage student academic records. The program reads a registration number, performs a linear search through an array of Student structures, and displays a formatted academic transcript. Through this lab, key C++ concepts were applied, including struct definition and initialization, nested structures, arrays of structures, and formatted console output using `iomanip`.

The exercise provided hands-on experience in designing structured data models and reinforced the concept that real-world data can be effectively represented using user-defined data types in C++.

10. References

9. Bjarne Stroustrup, *The C++ Programming Language, 4th Edition*, Addison-Wesley, 2013.
10. Stanley B. Lippman, *C++ Primer, 5th Edition*, Addison-Wesley Professional, 2012.
11. *cppreference.com – C++ Standard Library Reference*. <https://en.cppreference.com>
12. *EE170L Course Notes – C++ Programming Lab, UET, Semester Spring 2025*.